LAPORAN PRAKTIKUM
POSTTEST 7
ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Nama (2609106086)

Kelas (C1 ‘26)

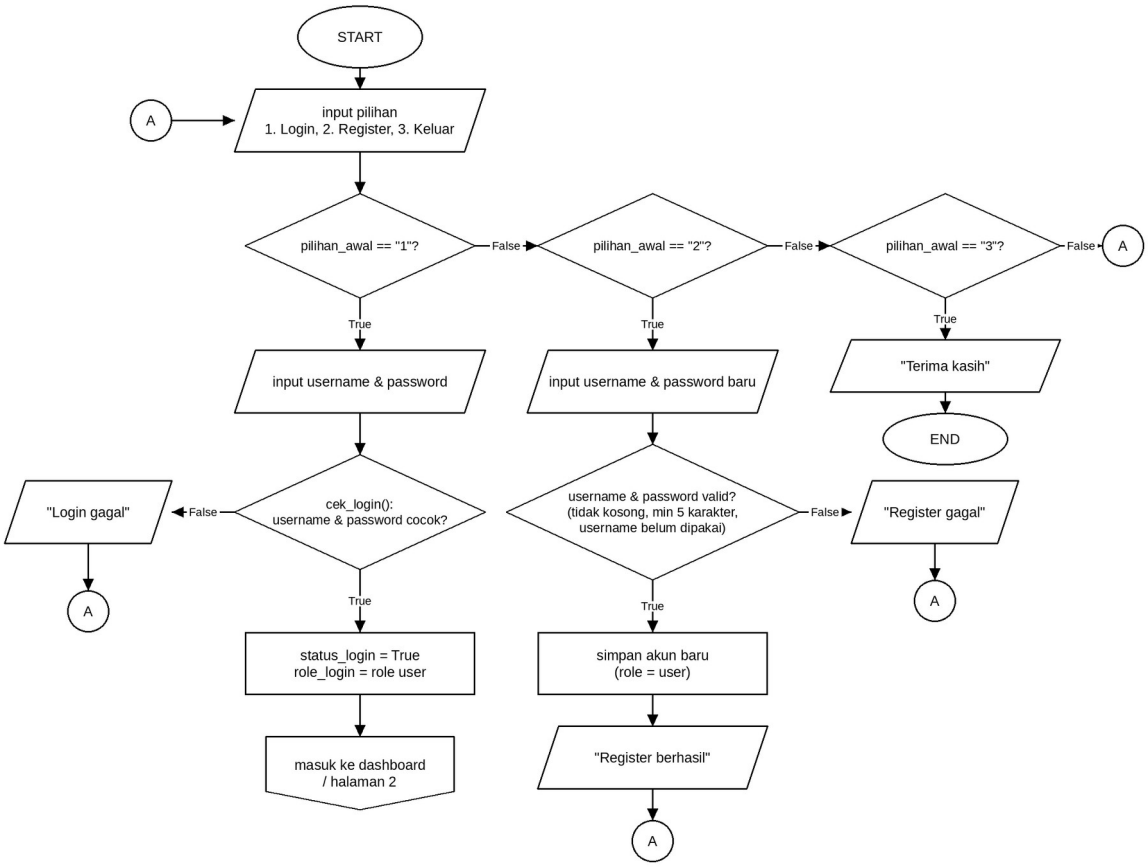
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

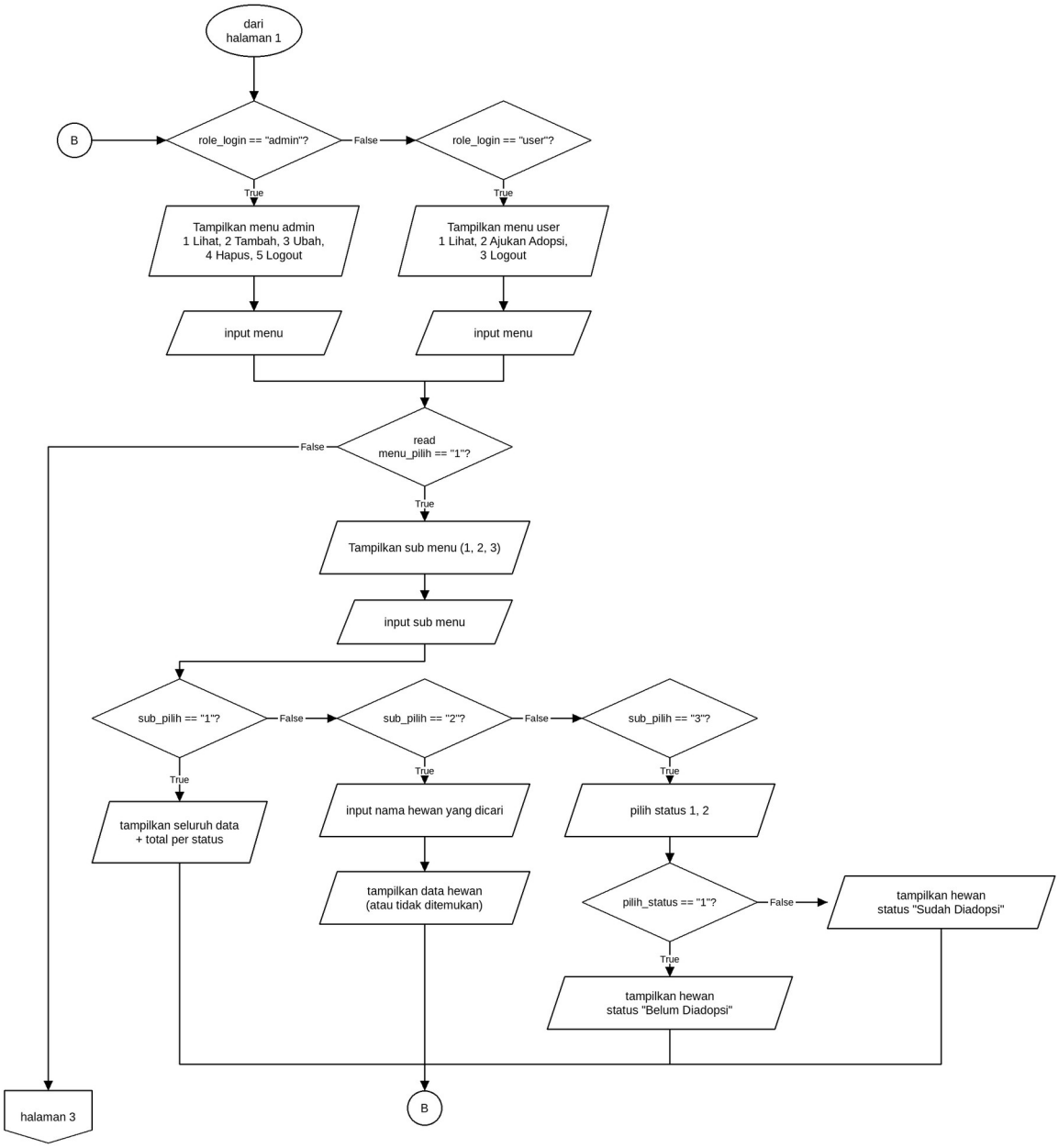
SAMARINDA

2026

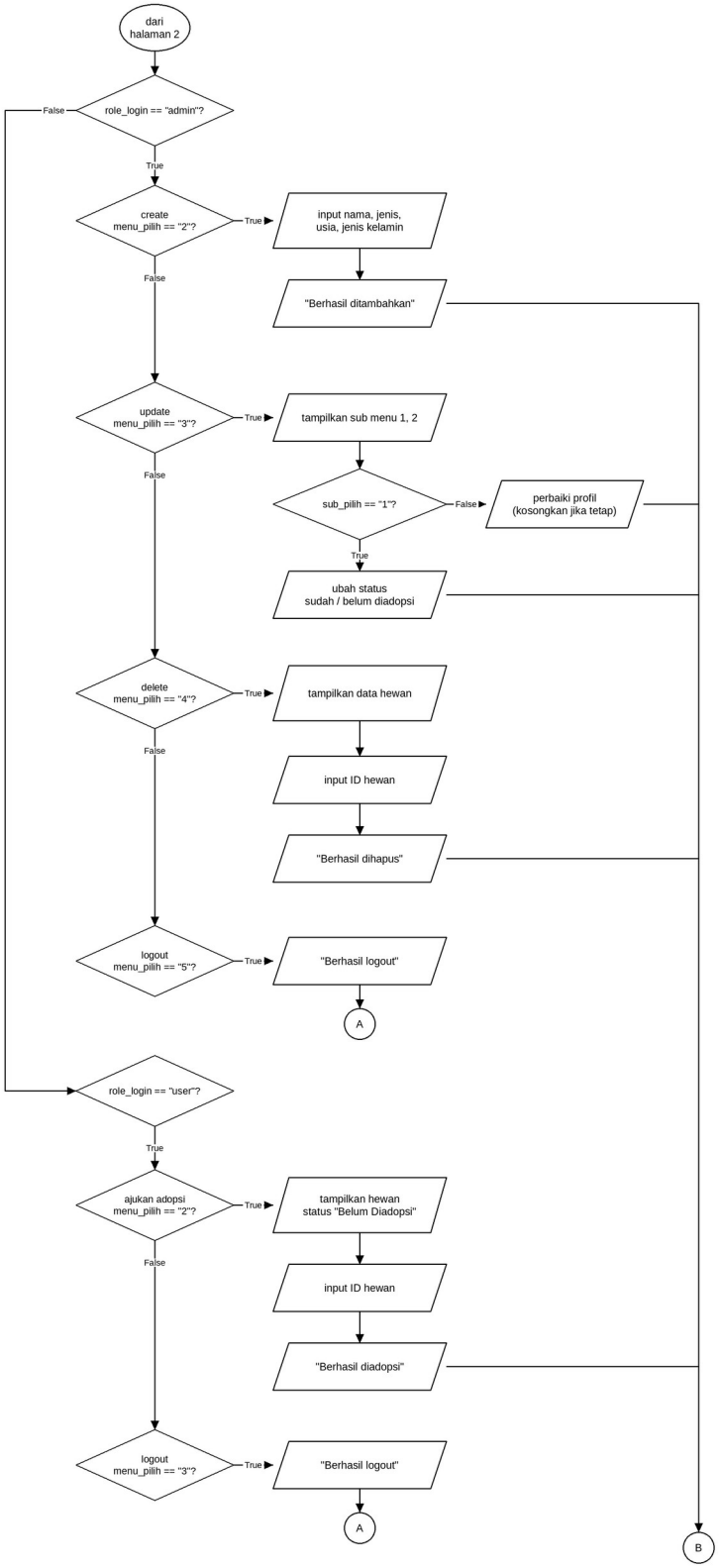
1. Flowchart



Gambar 1.1 Flowchart halaman 1



Gambar 1.2 Flowchart halaman 2



Gambar 1.3 Flowchart halaman 3

Keterangan: A = kembali ke menu awal (halaman 1); B = kembali ke menu dashboard (halaman 2).

2. Deskripsi Singkat Program

1. **Multiuser dan Register:** Membedakan peran login admin dan user, serta fitur register pembuatan akun baru yang langsung tersimpan ke dalam dictionary users. Register disertai validasi: username dan password tidak boleh kosong, minimal 5 karakter, dan username tidak boleh sama dengan yang sudah ada.
2. **CRUD:** Create, Read, Update, dan Delete pada data hewan peliharaan menggunakan dictionary dengan sistem key-value (ID hewan sebagai key). ID baru dibuat otomatis, dan fitur Read dilengkapi pencarian berdasarkan nama serta filter berdasarkan status adopsi.
3. **Fungsi dan Prosedur:** Program dipecah ke dalam fungsi (def). Fungsi yang mengembalikan nilai (return) antara lain generate_id_baru(), hitung_total_hewan(), input_angka(), cek_login(), dan ambil_hewan(). Prosedur yang tidak mengembalikan nilai antara lain bersihkan_layar(), tampilkan_header(), jeda(), dan cetak_hewan(). Parameter default dipakai pada jeda() dan cetak_hewan(), sedangkan status_login, role_login, dan user_aktif adalah variabel global yang diubah dengan kata kunci global.
4. **Fungsi Rekursif:** Fungsi hitung_status_rekursif() memanggil dirinya sendiri untuk menghitung jumlah hewan berdasarkan status adopsi, dan hasilnya ditampilkan pada menu Lihat Daftar Hewan.
5. **Exception Handling:** Menggunakan try-except-else-finally untuk menangani kesalahan input dan data, yaitu ValueError (input bukan angka dan validasi data dengan raise), KeyError (username atau ID hewan tidak ditemukan), serta KeyboardInterrupt dan EOFError pada fungsi main().

3. Source Code

A. Inisialisasi Data dan Fungsi Pendukung

Menginisialisasi penyimpanan data pengguna (users) dan data hewan (pets) berbasis dictionary, variabel global untuk status login, konstanta status adopsi, serta fungsi pendukung untuk membuat ID baru, menghitung total hewan, dan membaca input angka dengan try-except-else-finally.

Source Code:

```
import os

users = {
    "admin": ["admin123", "admin"],
    "alfie": ["123", "user"]
}

pets = {
    1: {"nama": "Milo", "jenis": "Kucing", "usia": "2 Tahun", "kelamin": "Jantan", "status":
"Belum Diadopsi"},
```

```
    2: {"nama": "Luna", "jenis": "Anjing", "usia": "1 Tahun", "kelamin": "Betina", "status":
"Belum Diadopsi"},
    3: {"nama": "Leo", "jenis": "Kucing", "usia": "3 Tahun", "kelamin": "Jantan", "status":
"Sudah Diadopsi"}
}

status_login = False
role_login = ""
user_aktif = ""

STATUS_BELUM = "Belum Diadopsi"
STATUS_SUDAH = "Sudah Diadopsi"

def generate_id_baru():
    if len(pets) == 0:
        return 1
    return max(pets.keys()) + 1

def hitung_total_hewan():
    total = len(pets)
    return total

def input_angka(pesan):
    try:
        angka = int(input(pesan))
    except ValueError:
        print("\n[ERROR] Input yang Anda masukkan bukan angka (integer)!")
        return None
    else:
        return angka
    finally:
        pass
```

B. Fungsi Login dan Register dengan Exception Handling

Fungsi cek_login() memakai try-except KeyError untuk memeriksa kredensial. Fungsi validasi memakai raise ValueError, yang ditangkap pada menu_register() dengan struktur try-except-else-finally.

Source Code:

```
def cek_login(username, password):
    try:
        data_user = users[username]
    except KeyError:
        return None
    else:
        if data_user[0] == password:
            return data_user[1]
        return None

def validasi_username(username):
    if username == "":
        raise ValueError("Username tidak boleh kosong.")
    if len(username) < 5:
        raise ValueError("Username minimal memiliki 5 karakter.")
    if username in users:
```

```
        raise ValueError("Username sudah digunakan, silakan gunakan yang lain.")

def validasi_password(password):
    if password == "":
        raise ValueError("Password tidak boleh kosong.")
    if len(password) < 5:
        raise ValueError("Password minimal memiliki 5 karakter.")

def menu_register():
    bersihkan_layar()
    print("--- REGISTER AKUN BARU ---")
    try:
        username_baru = input("Buat Username baru: ")
        validasi_username(username_baru)
        password_baru = input("Buat Password baru: ")
        validasi_password(password_baru)
    except ValueError as e:
        print(f"\n[GAGAL] {e}")
    else:
        users[username_baru] = [password_baru, "user"]
        print("\n[SUKSES] Akun berhasil didaftarkan! Silakan login.")
    finally:
        jeda("Tekan Enter untuk kembali ke menu utama...")
```

C. Fungsi Rekursif

Fungsi hitung_status_rekursif() menelusuri daftar ID hewan secara rekursif dengan indeks sebagai parameter, lalu dipanggil pada menu Lihat Daftar Hewan untuk menampilkan jumlah hewan per status.

Source Code:

```
def hitung_status_rekursif(daftar_id, status, indeks=0):
    if indeks == len(daftar_id):
        return 0
    cocok = 1 if pets[daftar_id[indeks]]["status"] == status else 0
    return cocok + hitung_status_rekursif(daftar_id, status, indeks + 1)
```

```
def lihat_hewan():
    ...
    if sub_read == "1":
        print("\n--- Seluruh Hewan ---")
        if hitung_total_hewan() == 0:
            print("Belum ada data hewan.")
        else:
            for id_pet, data in pets.items():
                cetak_hewan(id_pet, data)
            daftar_id = list(pets.keys())
            belum = hitung_status_rekursif(daftar_id, STATUS_BELUM)
            sudah = hitung_status_rekursif(daftar_id, STATUS_SUDAH)
            print(f"\nTotal: {hitung_total_hewan()} | {STATUS_BELUM}: {belum} |
{STATUS_SUDAH}: {sudah}")
```

D. Operasi CRUD dengan Dictionary

Dictionary pets menyimpan ID hewan sebagai key dan dictionary detail hewan sebagai value. Fungsi tambah_hewan() menambah data baru setelah validasi, sedangkan hapus_hewan() menghapus data dengan pets.pop() dan menangani KeyError jika ID tidak ditemukan.

Source Code:

```
def tambah_hewan():
    bersihkan_layar()
    print("--- TAMBAH DATA HEWAN BARU ---")
    id_baru = generate_id_baru()

    try:
        nama = input("Masukkan Nama Hewan: ")
        validasi_teks(nama, "Nama hewan")
        jenis = input("Masukkan Jenis Hewan (Kucing/Anjing): ")
        validasi_teks(jenis, "Jenis hewan")
        usia = input("Masukkan Usia Hewan (Contoh: 2 Tahun): ")
        validasi_teks(usia, "Usia hewan")
        kelamin = input("Masukkan Jenis Kelamin (Jantan/Betina): ")
        validasi_teks(kelamin, "Jenis kelamin")
        if kelamin.lower() not in ("jantan", "betina"):
            raise ValueError("Jenis kelamin harus 'Jantan' atau 'Betina'.")
    except ValueError as e:
        print(f"\n[GAGAL] {e}")
    else:
        pets[id_baru] = {
            "nama": nama,
            "jenis": jenis,
            "usia": usia,
            "kelamin": kelamin.capitalize(),
            "status": STATUS_BELUM
        }
        print(f"\n[SUKSES] Data hewan '{nama}' berhasil ditambahkan dengan ID {id_baru}!")
    finally:
        jeda()

def hapus_hewan():
    bersihkan_layar()
    print("--- HAPUS DATA HEWAN (DELETE) ---")
    for id_pet, data in pets.items():
        print(f"ID: {id_pet} | Nama: {data['nama']} | Status: {data['status']}")

    id_del = input_angka("\nMasukkan ID hewan yang ingin dihapus: ")
    if id_del is not None:
        try:
            hewan_terhapus = pets.pop(id_del)
        except KeyError:
            print("\n[GAGAL] ID hewan tidak ditemukan.")
        else:
            print(f"\n[SUKSES] Data hewan '{hewan_terhapus['nama']}' berhasil dihapus.")
    jeda()
```

E. Fungsi Utama (main)

Fungsi main() menampilkan menu awal secara berulang dan menangani penghentian program dengan KeyboardInterrupt dan EOFError, sedangkan blok finally selalu dijalankan saat program berakhir.

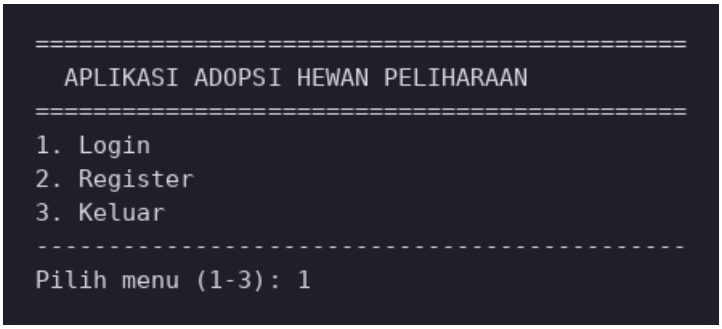
Source Code:

```
def main():
    try:
        while True:
            tampilkan_menu_awal()
            pilihan_awal = input("Pilih menu (1-3): ")

            if pilihan_awal == "1":
                menu_login()
                if status_login:
                    dashboard()
            elif pilihan_awal == "2":
                menu_register()
            elif pilihan_awal == "3":
                break
            else:
                print("\n[ERROR] Pilihan tidak valid.")
                jeda("Tekan Enter untuk melanjutkan...")
    except KeyboardInterrupt:
        print("\n\n[INFO] Program dihentikan paksa oleh pengguna (Ctrl+C).")
    except EOFError:
        print("\n\n[INFO] Input terputus, program dihentikan.")
    finally:
        print("Terima kasih telah menggunakan sistem adopsi hewan!")

if __name__ == "__main__":
    main()
```

4. Hasil Output



Gambar 4.1 Tampilan menu awal

```
--- MENU LOGIN ---
Masukkan Username: admin
Masukkan Password: admin123

[SUKSES] Selamat datang, admin (admin)!

Tekan Enter untuk masuk ke menu utama...
```

Gambar 4.2 Login sebagai admin

```
==============================================
  DASHBOARD ADOPSI HEWAN [ADMIN]
==============================================
1. Lihat Daftar Hewan
2. Tambah Data Hewan
3. Ubah Data / Status Hewan
4. Hapus Data Hewan
5. Logout
----------------------------------------------
Pilih menu: 1
```

Gambar 4.3 Dashboard admin

```
--- DAFTAR HEWAN PELIHARAAN ---
1. Tampilkan Semua Hewan
2. Cari Berdasarkan Nama
3. Filter Berdasarkan Status
Pilih opsi (1-3): 1

--- Seluruh Hewan ---
ID: 1 | Nama: Milo | Jenis: Kucing | Usia: 2 Tahun | Kelamin: Jantan | Status: Belum Diadopsi
ID: 2 | Nama: Luna | Jenis: Anjing | Usia: 1 Tahun | Kelamin: Betina | Status: Belum Diadopsi
ID: 3 | Nama: Leo | Jenis: Kucing | Usia: 3 Tahun | Kelamin: Jantan | Status: Sudah Diadopsi

Total: 3 | Belum Diadopsi: 2 | Sudah Diadopsi: 1

Tekan Enter untuk kembali...
```

Gambar 4.4 Lihat daftar hewan beserta total per status (fungsi rekursif)

```
--- TAMBAH DATA HEWAN BARU ---
Masukkan Nama Hewan: Bruno
Masukkan Jenis Hewan (Kucing/Anjing): Anjing
Masukkan Usia Hewan (Contoh: 2 Tahun): 1 Tahun
Masukkan Jenis Kelamin (Jantan/Betina): Jantan

[SUKSES] Data hewan 'Bruno' berhasil ditambahkan dengan ID 4!

Tekan Enter untuk kembali...
```

Gambar 4.5 Tambah data hewan

```
--- HAPUS DATA HEWAN (DELETE) ---
ID: 1 | Nama: Milo | Status: Belum Diadopsi
ID: 2 | Nama: Luna | Status: Belum Diadopsi
ID: 3 | Nama: Leo | Status: Sudah Diadopsi
ID: 4 | Nama: Bruno | Status: Belum Diadopsi

Masukkan ID hewan yang ingin dihapus: abc

[ERROR] Input yang Anda masukkan bukan angka (integer)!

Tekan Enter untuk kembali...
```

Gambar 4.6 Penanganan input bukan angka (exception handling)

```
--- REGISTER AKUN BARU ---
Buat Username baru: abc

[GAGAL] Username minimal memiliki 5 karakter.

Tekan Enter untuk kembali ke menu utama...
```

Gambar 4.7 Validasi username saat register (exception handling)

5. Git Add

[Tempel screenshot hasil perintah git add di sini]

```
git add .
```

Gambar 5.1 Git Add

Memasukkan file yang ada di direktori lokal ke dalam area git.

6. Git Commit

[Tempel screenshot hasil perintah git commit di sini]

```
git commit -m "upload: posttest apd 7"
```

Gambar 6.1 Git Commit

Menyimpan perubahan yang sudah berada di area git secara permanen.

7. Git Push

[Tempel screenshot hasil perintah git push di sini]

```
git push -u origin main
```

Gambar 7.1 Git Push

Mengunggah seluruh kode dan commit yang ada di *local repository* ke dalam *remote repository* di GitHub.